```
import zipfile
import os

zip_file_path = '/content/archive.zip'
extract_dir = '/content/'

if os.path.exists(zip_file_path):
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall(extract_dir)
    print(f"Successfully unzipped '{zip_file_path}' to '{extract_dir}'")
else:
    print(f"Error: The file '{zip_file_path}' was not found.")
```

```
Successfully unzipped '/content/archive.zip' to '/content/'
```

```
import pandas as pd

# Load the dataset
df = pd.read_csv('/content/insurance.csv')

# Identify categorical columns to encode
categorical_cols = ['sex', 'smoker', 'region']

# Apply one-hot encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("Original DataFrame head:")
print(df.head())
print("\nEncoded DataFrame head:")
print(df_encoded.head())
print("\nEncoded DataFrame info:")
df_encoded.info()
```

```
Original DataFrame head:
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
```

```
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520

Encoded DataFrame head:
   age     bmi  children      charges  sex_male  smoker_yes  region_northwest  \
0   19  27.900         0  16884.92400     False        True             False
1   18  33.770         1   1725.55230      True       False             False
2   28  33.000         3   4449.46200      True       False             False
3   33  22.705         0  21984.47061      True       False              True
4   32  28.880         0   3866.85520      True       False              True

   region_southeast  region_southwest
0             False             True
1              True            False
2              True            False
3             False            False
4             False            False

Encoded DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   age               1338 non-null   int64
 1   bmi               1338 non-null   float64
 2   children          1338 non-null   int64
 3   charges           1338 non-null   float64
 4   sex_male          1338 non-null   bool
 5   smoker_yes        1338 non-null   bool
 6   region_northwest  1338 non-null   bool
 7   region_southeast  1338 non-null   bool
 8   region_southwest  1338 non-null   bool
dtypes: bool(5), float64(2), int64(2)
memory usage: 48.5 KB
```

```
features = ['age', 'bmi', 'children', 'smoker_yes']
target = 'charges'
```

```
X = df_encoded[features]
y = df_encoded[target]

print("Features (X) head:")
print(X.head())
print("\nTarget (y) head:")
print(y.head())
```

```
Features (X) head:
   age     bmi  children  smoker_yes
0   19  27.900         0        True
1   18  33.770         1       False
2   28  33.000         3       False
3   33  22.705         0       False
4   32  28.880         0       False

Target (y) head:
0    16884.92400
1     1725.55230
2     4449.46200
3    21984.47061
4     3866.85520
Name: charges, dtype: float64
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("--- Training Linear Regression Model ---")
# Initialize and train the Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_linear = linear_model.predict(X_test)
```

```
# Evaluate the Linear Regression model
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print(f"Linear Regression MSE: {mse_linear:.2f}")
print(f"Linear Regression R-squared: {r2_linear:.2f}")

print("\n--- Training Ridge Regression Model ---")
# Initialize and train the Ridge Regression model
# Using an alpha value, which controls the strength of regularization
# A common practice is to tune this parameter (e.g., using GridSearchCV)
ridge_model = Ridge(alpha=1.0) # You can experiment with different alpha values
ridge_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_ridge = ridge_model.predict(X_test)

# Evaluate the Ridge Regression model
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print(f"Ridge Regression MSE: {mse_ridge:.2f}")
print(f"Ridge Regression R-squared: {r2_ridge:.2f}")
```

```
--- Training Linear Regression Model ---
Linear Regression MSE: 33981653.95
Linear Regression R-squared: 0.78

--- Training Ridge Regression Model ---
Ridge Regression MSE: 34019336.82
Ridge Regression R-squared: 0.78
```

```
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

alpha_values = [0.1, 1, 10, 100]
```

```
print("--- Ridge Regression with Different Alpha Values ---")

for alpha in alpha_values:
    print(f"\nTraining Ridge Regression with alpha = {alpha}")

    # Initialize and train the Ridge Regression model
    ridge_model = Ridge(alpha=alpha)
    ridge_model.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred_ridge = ridge_model.predict(X_test)

    # Evaluate the Ridge Regression model
    mse_ridge = mean_squared_error(y_test, y_pred_ridge)
    r2_ridge = r2_score(y_test, y_pred_ridge)

    print(f"Ridge Regression (alpha={alpha}) MSE: {mse_ridge:.2f}")
    print(f"Ridge Regression (alpha={alpha}) R-squared: {r2_ridge:.2f}")
```

```
--- Ridge Regression with Different Alpha Values ---

Training Ridge Regression with alpha = 0.1
Ridge Regression (alpha=0.1) MSE: 33985173.33
Ridge Regression (alpha=0.1) R-squared: 0.78

Training Ridge Regression with alpha = 1
Ridge Regression (alpha=1) MSE: 34019336.82
Ridge Regression (alpha=1) R-squared: 0.78

Training Ridge Regression with alpha = 10
Ridge Regression (alpha=10) MSE: 34580366.51
Ridge Regression (alpha=10) R-squared: 0.78

Training Ridge Regression with alpha = 100
Ridge Regression (alpha=100) MSE: 48319004.79
Ridge Regression (alpha=100) R-squared: 0.69
```

```
import pandas as pd

# Collect the results
```

```
results = {
    'Model': ['Linear Regression', 'Ridge (alpha=0.1)', 'Ridge (alpha=1)', '
    'MSE': [mse_linear, 33985173.33, 34019336.82, 34580366.51, 48319004.79],
    'R-squared': [r2_linear, 0.78, 0.78, 0.78, 0.69]
}

results_df = pd.DataFrame(results)
print("\n--- Model Performance Comparison ---")
print(results_df)

# Identify the best Ridge model based on R-squared (higher is better) and th
best_ridge_alpha_index = results_df[results_df['Model'].str.contains('Ridge'
best_ridge_model = results_df.loc[best_ridge_alpha_index]

print(f"\n--- Best Ridge Model ---")
print(f"The best performing Ridge model has an alpha of: {best_ridge_model['
print(f"MSE: {best_ridge_model['MSE']:.2f}")
print(f"R-squared: {best_ridge_model['R-squared']:.2f}")

# Overall best model comparison (including Linear Regression)
overall_best_model_index = results_df.sort_values(by=['R-squared', 'MSE'], a
overall_best_model = results_df.loc[overall_best_model_index]

print(f"\n--- Overall Best Model (Linear or Ridge) ---")
print(f"The overall best performing model is: {overall_best_model['Model']}"
print(f"MSE: {overall_best_model['MSE']:.2f}")
print(f"R-squared: {overall_best_model['R-squared']:.2f}")
```

```
--- Model Performance Comparison ---
               Model           MSE  R-squared
0  Linear Regression  3.398165e+07   0.781115
1  Ridge (alpha=0.1)  3.398517e+07   0.780000
2    Ridge (alpha=1)  3.401934e+07   0.780000
3   Ridge (alpha=10)  3.458037e+07   0.780000
4  Ridge (alpha=100)  4.831900e+07   0.690000

--- Best Ridge Model ---
The best performing Ridge model has an alpha of: 0.1
MSE: 33985173.33
```

```
R-squared: 0.78

--- Overall Best Model (Linear or Ridge) ---
The overall best performing model is: Linear Regression
MSE: 33981653.95
R-squared: 0.78
```